Empiria: funcionamiento de la implementación con Alpine.js

Guía de exposición y defensa del código · CU-002 y CU-005

6 de noviembre de 2026

1. Qué exige la entrega y dónde se cumple
2. Arquitectura: qué ejecuta cada archivo
   Cómo arranca la aplicación
3. Navegación: estado compartido y vistas visibles
4. Puente asíncrono con Google Apps Script
5. CU-002: del formulario al resultado del servidor
   5.1 Cargar y presentar plantillas
   5.2 Formulario y publicación
   5.3 Reintento, doble envío y cancelación
6. CU-005: respuesta, filtros y filas reactivas
   6.1 Consulta y recuperación ante errores
   6.2 Selectores y tablas
7. Preguntas para defender el código
8. Recorrido para la exposición y verificaciones

1. Qué exige la entrega y dónde se cumple

La actualización tecnológica exige usar Alpine.js para navegar entre vistas, mostrar y restaurar estados de carga durante llamadas al servidor, y poblar HTML con respuestas del servidor en al menos una funcionalidad de cada caso de uso.

Criterio	CU-002: generar actividad	CU-005: generar reporte
Navegación	Store `navegacion` y `x-show` para entrar en Tareas	Mismo store para entrar en Reportes y cambiar investigador/carrera
Carga reactiva	`cargandoPlantillas`, `publicando` y `cancelando`	`cargando` y `abriendoConsolidado`
Poblamiento	Catálogo del servidor recorrido con `x-for` en el selector de plantillas	Respuesta del servidor convertida en selectores y filas de publicaciones con `x-for`

Alcance: en Apps Script las llamadas `google.script.run` son reales. La publicación de CU-002 sigue siendo una simulación ejecutada en el servidor y los reportes usan datos de demostración. El backend experimental de registro real con Classroom es independiente y no está conectado a estos bo-

tones. El login también es simulado: no constituye autenticación ni autorización.

2. Arquitectura: qué ejecuta cada archivo

Archivo	Responsabilidad y lugar de ejecución
`Código.gs`	Servidor: `doGet()` abre la aplicación e `include()` compone HTML
`TareasServidor.gs`	Servidor: catálogo, validación, publicación demo y cancelación
`ReportesServidor.gs`	Servidor: consulta y validación de datos de reportes
`Consolidados.gs`	Servidor: consulta o creación del consolidado en Sheets
`BaseDatos.gs`	Servidor: define `BASE_DATOS_EMPIRIA` con datos demo
`Index.html`	Compone vistas, carga Alpine y habilita su ámbito con `x-data`
`Navegacion.html`	Navegador: registra el store compartido de navegación
`Reactivos.html`	Navegador: adapta `google.script.run` a promesas
`AlpineTareas.html`	Navegador: estado del formulario y carga de plantillas
`AlpineReportes.html`	Navegador: carga, filtros, selectores y tablas de reportes
`Tareas.html`, `Reportes.html` y subreportes	Estructura de las vistas y directivas Alpine
`TablaPublicaciones.html`	Tabla compartida por investigador y carrera
`FlujosTareas.html`	Transacciones, respuestas, reintentos y cancelación de CU-002
`Scripts.html`	Funciones de apoyo, cálculos, filtros, gráficos y exportaciones
`Estilos.html`	Estilos y dependencias visuales

Los `.gs` se ejecutan en Google; los bloques `<script>` de los HTML se ejecutan en el navegador. Alpine pertenece al navegador, por eso sus módulos son HTML con JavaScript dentro. Apps Script comparte las funciones de los archivos `.gs`: no necesitan importarse entre sí. Se mantienen nombres base distintos para servidor y vistas, como `TareasServidor` y `Tareas`.

Cómo arranca la aplicación

1. Google ejecuta `doGet()` en `Código.gs`.
2. `HtmlService.createTemplateFromFile('Index').evaluate()` procesa la plantilla.
3. Las expresiones `include('...')` insertan los componentes. `include()` también evalúa plantillas, por lo que funcionan las inclusiones anidadas de Reportes.

4. El navegador recibe el HTML y carga Alpine 3.17.5 con `defer`.
5. Los módulos registran escuchas de `alpine:init`; cuando Alpine se inicializa, crean los stores `navegacion`, `tareas` y `reportes`.
6. Alpine interpreta las directivas dentro del ámbito `x-data` del documento.

Diferencia importante: `include()` compone el documento antes de enviarlo. `x-for` crea y actualiza elementos en el navegador a partir del estado reactivo.

3. Navegación: estado compartido y vistas visibles

En `Navegacion.html`, `Alpine.store('navegacion', {...})` guarda `vistaActual`, `reporteActual` y `sesionIniciada`. Las vistas leen ese mismo estado con `$store`. Este fragmento real de Reportes determina su visibilidad:

```
        x-show="$store.navegacion.vistaActual === 'reportes'"
```

El método `navegar(destino)` comprueba que el destino exista en `MODULOS` y asigna `this.vistaActual = destino`. La asignación dispara la actualización reactiva: Alpine muestra la vista seleccionada y oculta las otras sin solicitar otra página.

Después usa `Alpine.nextTick(...)` para ejecutar la acción de entrada cuando el DOM ya refleja la vista visible. Esto permite que los gráficos midan un contenedor visible. También comprueba que el usuario no haya cambiado de destino entretanto.

`cambiarReporte(nombre)` solo acepta `investigador` o `carrera` y cambia `reporteActual`. El login usa `x-show` según `sesionIniciada`.

Por qué `x-show`: cambia la visibilidad y conserva el DOM, los controles y su estado. No destruye y reconstruye toda la pantalla en cada navegación. `x-cloak`, en las vistas que lo usan, evita mostrarlas antes de inicializar Alpine.

4. Puente asíncrono con Google Apps Script

`google.script.run` funciona con callbacks: el servidor responde posteriormente. En `Reactivos.html`, `llamarServidor(nombre, argumentos)` lo envuelve en una promesa. Este es el núcleo del código:

```
        var runner = google.script.run
          .withSuccessHandler(resolve)
          .withFailureHandler(reject);
        runner[nombre].apply(
          runner,
          JSON.parse(JSON.stringify(argumentos || []))
        );
```

El nombre selecciona el endpoint público; `apply` pasa la lista de argumentos. `resolve` entrega la respuesta y `reject` comunica el error. La serialización convierte los parámetros usados en estas llamadas en datos JSON simples.

Esto permite escribir `await llamarServidor(...)`: la función asíncrona espera la promesa, mientras el navegador continúa atendiendo su interfaz. Si no existe el puente de Google, el adaptador rechaza la petición con un error explícito; no inventa una respuesta de éxito.

La respuesta no está disponible inmediatamente después de iniciar la llamada. Por eso los estados de carga se activan antes de esperar y se restauran al terminar.

5. CU-002: del formulario al resultado del servidor

5.1 Cargar y presentar plantillas

El store `tareas` contiene el formulario, el catálogo y sus estados de carga/error. `cargarPlantillas()` sigue este recorrido:

1. Obtiene el ciclo y evita repetir una consulta ya resuelta o en curso para él.
2. Incrementa `solicitud`, activa `cargandoPlantillas` y limpia `errorPlantillas`.
3. Ejecuta `await llamarServidor('obtenerPlantillasTarea', [ciclo])`.
4. Valida que la respuesta corresponda al ciclo y tenga plantillas con estructura válida. Conserva las asociaciones y plantillas locales de la demostración.
5. Actualiza el catálogo y los datos del store usados por la vista.
6. En `catch` conserva un mensaje de error; en `finally` restaura la carga si sigue siendo la solicitud vigente.

El identificador `solicitud` evita que una respuesta antigua sustituya los datos de una petición más reciente. El getter `disponibles` filtra las plantillas cuyo estado permite seleccionarlas. `Tareas.html` las presenta con este código:

```
        <template x-for="plantilla in $store.tareas.disponibles"
                  :key="plantilla.id">
          <option :value="plantilla.id"
                  x-text="plantilla.nombre + plantilla.extension"></option>
        </template>
```

Aquí se cumple el poblamiento exigido: la consulta al servidor alimenta el estado y `x-for` genera las opciones. `:key` identifica cada elemento; `x-text` escribe texto sin interpretarlo como HTML.

5.2 Formulario y publicación

Los controles usan `x-model` sobre `$store.tareas.formulario`: lo escrito actualiza el estado y los cambios en ese estado se reflejan en los controles. `crearTarea(e)` en `Scripts.html` evita el envío tradicional, valida los campos y pasa al flujo de publicación. `setBotonCreando(true)` asigna `publicando = true` en el store.

En `Tareas.html` esa propiedad gobierna `:disabled`, el icono y el texto:

```
        :disabled="$store.tareas.publicando"
        x-text="$store.tareas.publicando ? 'Creando tarea…' : 'Crear tarea'"
```

`FlujosTareas.html` coordina la transacción y `procesarPublicacion()` llama a `publicarActividadDemo` mediante el adaptador. Guarda la respuesta o el error pendiente y `resolverRespuestaPublicacion()` decide cómo finalizar.

El endpoint en `TareasServidor.gs` vuelve a validar ciclo, título, descripción, fecha de Lima, plantilla y rango de hojas. **La validación del navegador mejora la interacción, pero el servidor debe validar sus propios parámetros.**

El resultado exitoso contiene `idOperacion`, `idTarea`, estado y `simulacion: true`. El flujo valida la respuesta, actualiza la transacción y restaura los controles. Ante un error definitivo, conserva los datos para corregir o reintentar.

5.3 Reintento, doble envío y cancelación

- `idOperacion` identifica la misma operación durante los reintentos. El servidor consulta su registro demo y devuelve un resultado previo si ya existe.
- `conRegistroDemo_()` utiliza bloqueo y caché de usuario para coordinar acceso. Ese caché es temporal: no equivale a persistencia de producción.
- Los errores de cuota admiten tres reintentos con espera progresiva. El formulario permanece en proceso hasta resolver el resultado final.
- La cancelación llama a `cancelarActividadDemo`. `cancelando` evita duplicar esa acción y se restaura en `finally`.
- Una respuesta tardía no confirma una transacción que ya fue cancelada. Si la cancelación está pendiente, el flujo espera su resultado antes de confirmar éxito.

No toda la publicación se resuelve con un `finally` sencillo: hay coordinación de transacciones, reintentos y cancelación. Alpine refleja el estado resultante.

6. CU-005: respuesta, filtros y filas reactivas

6.1 Consulta y recuperación ante errores

`AlpineReportes.html` registra el store `reportes`. Su método `cargar(forzar)` activa `cargando`, limpia el error y oculta los paneles de resultados. Después:

```
        var respuesta = await llamarServidor(
          'obtenerDatosReportes',
          [{ condicion: condicion }]
        );
```

`ReportesServidor.gs` obtiene `BASE_DATOS_EMPIRIA`, valida investigadores y sus colecciones y devuelve `{ investigadores: [...] }`. No ejecuta los filtros de la tabla: esos filtros se aplican posteriormente en el navegador.

El cliente valida el contrato, asigna `this.datos = respuesta` y llama a `renderizarVistaReportes(respuesta)`. Tras `Alpine.nextTick()` muestra los paneles. Si falla, borra los datos del store y presenta el error; `finally` desactiva `cargando`. La vista puede ofrecer Reintentar sin quedar bloqueada.

6.2 Selectores y tablas

Los selectores guardan sus opciones en `opciones[id]`, la selección en `filtros[id]` y su estado vacío en `sinOpciones[id]`. `llenarSelector()` actualiza esas estructuras; Alpine construye las opciones con `x-for` y mantiene la selección con `x-model`.

Para publicaciones, las funciones de `Scripts.html` calculan búsqueda, filtros, orden y página. `renderizarColeccionInv(tipo)` transforma el resultado en datos:

```
var tabla = Alpine.store('reportes').tablas[tipo];
tabla.columnas = columnasVisiblesColeccionInv(tipo, pagina.visibles);
tabla.filas = pagina.visibles.map(function(item, indice) {
  return {
    clave: String(item.id || item.titulo || 'registro') +
      '-' + (pagina.inicio + indice),
    item: item
  };
});
```

`TablaPublicaciones.html` consume ese estado:

```
<template x-for="fila in $store.reportes.tablas[tipoPublicaciones].filas"
          :key="fila.clave">
  <tr>
    <!-- Un x-for interior genera las celdas de las columnas visibles. -->
  </tr>
</template>
```

La vista se comparte entre publicaciones individuales y de carrera. Cambiar la búsqueda recalcula las filas, actualiza el store y Alpine actualiza la tabla. **Buscar en la tabla no hace otra consulta al servidor:** trabaja con el conjunto ya recibido. La carga inicial es la funcionalidad que obtiene esos datos remotos.

Las celdas que usan `x-html` pasan por `celdaColeccionInv`, que escapa texto y valida enlaces. No debe colocarse directamente HTML arbitrario recibido del servidor. Los gráficos y otros listados conservan renderizadores tradicionales; no toda la interfaz fue migrada. Las tablas principales de publicaciones y los selectores migrados son propiedad de Alpine y se actualizan modificando estado.

7. Preguntas para defender el código

Pregunta	Respuesta centrada en esta implementación
¿Por qué Alpine y también Scripts.html?	Alpine controla estado y presentación migrada; las funciones JavaScript calculan, validan, exportan y coordinan operaciones. Alpine también es JavaScript.
¿Qué diferencia hay entre x-show y x-for?	x-show cambia visibilidad; x-for genera elementos a partir de una colección.
¿Para qué x-model?	Sincroniza el valor del control con el estado del formulario o filtro.

Pregunta	Respuesta centrada en esta implementación
¿Qué significan :disabled y @click?	Son abreviaturas de x-bind:disabled y x-on:click: enlazar un atributo y atender un evento.
¿Por qué un store?	Comparte estado entre componentes incluidos en distintos HTML, como botones, formularios y paneles.
¿Qué hace finally?	Restaura la carga aunque el servidor falle; CU-002 además coordina transacciones y reintentos.
¿Por qué nextTick?	Espera la actualización del DOM antes de mostrar paneles o medir gráficos.
¿Por qué no innerHTML en la tabla migrada?	Alpine ya gestiona sus nodos con x-for; se actualizan los arrays del store.
¿La búsqueda siempre consulta a Google?	No. La consulta inicial obtiene el conjunto; búsqueda y filtros de publicaciones son locales.
¿Cambiar un archivo .gs requiere imports?	No. Comparten el espacio de funciones del proyecto Apps Script.
¿Publicamos realmente en Classroom desde el botón?	No. El endpoint devuelve una publicación demo. La integración real está separada y pendiente.

8. Recorrido para la exposición y verificaciones

1. Abrir la URL de Apps Script actualizada e iniciar la sesión demo.
2. Navegar entre Inicio, Tareas y Reportes y explicar store + `x-show`.
3. En Tareas, señalar la consulta de plantillas y su `x-for`. Completar una tarea ficticia con fecha futura dentro del ciclo 2026-II y rango válido.
4. Crear la actividad y mostrar texto de carga, controles bloqueados y resultado.
5. Entrar en Reportes y mostrar carga, selectores y publicaciones. Cambiar el investigador, buscar y explicar respuesta → store → filas → `x-for`.
6. Cambiar a carrera para mostrar la tabla compartida. Si se demuestra un error, restaurar la condición normal y comprobar la recuperación.

El código cuenta con 27 pruebas de servidor y 16 escenarios de navegador que comprueban carga, respuestas, errores, reintentos, cancelación, navegación y tablas. Las pruebas locales usan Alpine real y dobles de los servicios Google; **no sustituyen la comprobación en la URL final de Apps Script**.

Para instalar, seguir `docs/ENTREGA_ALPINE.md`. La versión modular tiene cinco archivos `.gs` y veinte `.html`, además del manifiesto y registro ya conservados. La URL `/dev` utiliza el código guardado; `/exec` requiere actualizar la versión de la implementación para presentar los cambios nuevos.